

A Categorical Analysis and Refinement of the BreakDex "Add Move" End-to-End Flow

Section 1: A Categorical Model of the "Add Move" Flow

This section establishes a formal model of the "Add Move" feature using the language of Category Theory. By abstracting the system's components and operations into a categorical framework, we can reason about its structure, identify points of failure with mathematical precision, and verify the correctness of proposed solutions. This approach moves beyond ad-hoc debugging to a first-principles analysis of the system's architecture.

1.1 Introduction to Applied Category Theory for Software Systems

Category Theory is a branch of mathematics that formalizes structure and structure-preserving transformations.¹ Its power lies in its ability to abstract away implementation details to reveal underlying patterns and relationships.² In the context of software engineering, we can model a system as a

category, which consists of two primary components: **objects** and **morphisms** (also called arrows).¹

- **Objects:** These represent the types, data structures, or states within the system. For example, a Swift String, an AVAsset, or the state of a UI can be considered objects in our category.
- **Morphisms:** These represent the functions, processes, or transformations that map one object to another. A function $(\text{Int}) \rightarrow \text{String}$ is a morphism from the Int object to the String object.
- **Composition:** Morphisms can be composed. If there is a morphism $f: A \rightarrow B$ and another $g: B \rightarrow C$, their composition $g \circ f$ is a morphism from $A \rightarrow C$. In software, this corresponds

to the sequential application of functions.⁶

The central tool for verifying correctness in this model is the **commutative diagram**. A diagram is a collection of objects and morphisms. It is said to "commute" if any two paths between the same starting and ending objects are equal.⁹ In software, this means that different sequences of operations intended to achieve the same result must actually produce identical outcomes.⁷ A bug, particularly one involving state transformation, can often be modeled as a diagram that fails to commute.

To facilitate this analysis, we define two distinct categories:

- **CIdeal**: The category representing the *specified, error-free* system. In CIdeal, all diagrams commute by definition, as it represents the desired behavior.
- **CSystem**: The category representing the *current, buggy* implementation of the BreakDex application. The issues reported by the user indicate that certain diagrams in this category do not commute.

The process of debugging can be framed as defining a **functor**, $F: \text{CIdeal} \rightarrow \text{CSystem}$. A functor is a structure-preserving map between categories; it maps objects to objects and morphisms to morphisms while respecting composition and identity.¹ The bugs in our system suggest that our implementation functor,

F , is "unfaithful" or "forgetful"—it fails to preserve the commutative properties of CIdeal.¹³ The proposed fixes will be presented as

natural transformations, which are morphisms between functors.¹ A natural transformation provides a systematic way to modify our functor

F into a corrected functor F' such that the diagrams in CSystem now commute, bringing the implementation in line with the specification.

1.2 The Objects of the "Add Move" Category

We formally define the primary data types and application states as the objects within our category, CSystem. This abstraction allows us to reason about the system's flow as a path through these states.

- **VideoFile**: Represents the initial state, an unprocessed video file identified by a URL.
- **AVAssetState**: Represents a video that has been loaded into memory as an AVAsset object, making its tracks and metadata available for processing.
- **TrimmedState**: An AVAssetState that has been associated with a specific time range,

CMTIMERange, representing the user's trim selection. This is an object composed of (AVAsset, CMTIMERange).

- **RotatedState:** A TrimmedState that has been associated with a user-defined rotation angle (e.g., 90, 180, 270 degrees). This object is composed of (AVAsset, CMTIMERange, Angle).
- **ExportingState:** Represents an in-progress video processing operation, encapsulated by an AVAssetExportSession. This is a transient state object.
- **FinalAsset:** Represents the successfully processed video, saved to a temporary file URL on disk, ready for persistence.
- **PHAssetState:** The state of the video after it has been saved to the user's Photos library, represented by a PHAsset object.
- **PHAlbumState:** Represents the target "BreakDex" album in the Photos library, an object of type PHAssetCollection.
- **CoreDataMoveState:** The final persisted state in the local database, represented by a Move entity in CoreData.
- **UIState:** The collective state of the user interface, including progress indicators, view visibility, and user-interactive elements.

1.3 The Morphisms of the "Add Move" Category

We define the key operations and data flows as morphisms (arrows) connecting the objects defined above. These morphisms represent the state transitions within the application.

- loadAsset: VideoFile $\rightarrow$ AVAssetState
- applyTrim: AVAssetState $\times$ CMTIMERange $\rightarrow$ TrimmedState
- applyRotation: TrimmedState $\times$ Angle $\rightarrow$ RotatedState
- exportVideo: RotatedState $\rightarrow$ FinalAsset (This is a complex, asynchronous morphism)
- saveToPhotos: FinalAsset $\rightarrow$ PHAssetState
- findOrCreateAlbum: String $\rightarrow$ PHAlbumState
- addToAlbum: PHAssetState $\times$ PHAlbumState $\rightarrow$ PHAlbumState
- saveToCoreData: PHAssetState $\times$ String $\rightarrow$ CoreDataMoveState

1.4 The E2E Flow as a Composition of Morphisms

The entire "add move" workflow, from selecting a video to saving the final named move, can be modeled as a single, long composition of these morphisms. The user's goal for a "fully, functional, correct, precise E2E add move flow" is a formal requirement that this composite

morphism be well-defined, deterministic, and free of side effects that violate the system's integrity.

The composition can be expressed as:

```
saveToCoreData◦(addToAlbum◦(saveToPhotos◦exportVideo◦applyRotation◦applyTrim◦loadAs  
set))
```

The reported bugs indicate that this composition is currently flawed. Specifically, the analysis will show that the `applyRotation` morphism introduces visual distortions, and the composition of `findOrCreateAlbum` and `addToAlbum` is not idempotent, leading to data corruption in the Photos library. A failure in any single morphism breaks the entire compositional chain, invalidating the end-to-end result. The categorical approach allows us to decompose this complex flow and analyze each morphism in isolation to pinpoint the exact source of failure, rather than treating the E2E process as an inscrutable monolith.²

To provide a clear reference for the subsequent analysis, the following table formalizes the mapping between the abstract categorical components and their concrete Swift implementations. This table serves as the foundational dictionary for translating between the formal model and the codebase.

Table 1: System Category Objects and Morphisms

Category Element	Type	Concrete Swift Implementation
Objects		
VideoFile	State	URL pointing to a video file.
AVAssetState	State	AVAsset instance.
TrimmedState	State	Tuple: (asset: AVAsset, timeRange: CMTimeRange).
RotatedState	State	Tuple: (asset: AVAsset, timeRange: CMTimeRange, rotation: CGFloat).
ExportingState	State	AVAssetExportSession instance.

FinalAsset	State	URL pointing to the processed, temporary video file.
PHAssetState	State	PHAsset instance.
PHAlbumState	State	PHAssetCollection instance for "BreakDex" album.
CoreDataMoveState	State	Move managed object instance in a NSManagedObjectContext.
UIState	State	The state of the relevant SwiftUI Views (e.g., @State variables).
Morphisms		
loadAsset	Function	AVAsset(url: videoURL).
applyTrim	Function	Logic that stores the selected CMTimeRange with the AVAsset.
applyRotation	Function	Logic that applies a transform to the video, producing an AVMutableVideoComposition.
exportVideo	Function	AVAssetExportSession.exportAsynchronously(completionHandler:).
saveToPhotos	Function	PHPhotoLibrary.performChanges block with PHAssetChangeRequest.creationRequestForAssetFrom

		mVideo.
findOrCreateAlbum	Function	Logic that fetches or creates a PHAssetCollection by title.
addToAlbum	Function	PHAssetCollectionChangeRequest.addAssets(_:).
saveToCoreData	Function	Logic to create a Move entity and save the NSManagedObjectContext.

Section 2: Analysis of Video Transformation

Morphisms: Correcting Rotation and Aspect Ratio

This section investigates the `applyRotation` morphism, which is responsible for the "cropped square effect." The analysis will demonstrate that this visual artifact is a direct result of an incorrectly constructed affine transformation that fails to account for the video's intrinsic geometry. We will derive the correct composite transformation from first principles of linear algebra and AVFoundation's architecture, ensuring aspect-ratio preservation.

2.1 The Flawed `applyRotation` Morphism

The observed cropping indicates that the current implementation, `applyRotation_{buggy}`, is not a geometrically sound transformation. It appears to apply a rotation without correctly adjusting the rendering canvas or the video's position within it. This results in a morphism that is not an **isomorphism** on the video's visual space. An isomorphism is a morphism that has an inverse; it is a structure-preserving transformation that can be perfectly undone.² The

`applyRotation_{buggy}` morphism discards pixel data (the cropped portions), meaning information is lost. This loss of information makes the operation non-invertible and thus, not an isomorphism.

The goal is to construct a new morphism, `applyRotation_{correct}`, that is an isomorphism. This requires a transformation that maps the original pixel space to the new, rotated pixel space without loss of data, preserving the aspect ratio. This is achieved by using `AVMutableVideoComposition` and `AVMutableVideoCompositionLayerInstruction` to define a transformation that respects the video's `naturalSize` and `preferredTransform`.

2.2 Deconstructing `AVAssetTrack.preferredTransform`

A primary source of error in video rotation is a misunderstanding of the `AVAssetTrack.preferredTransform` property. This is not a simple angle but a `CGAffineTransform` matrix that encodes the orientation metadata written by the recording device.¹⁵ The matrix has the form:

`ac0bd0txty1`

The components (a, b, c, d) define a linear transformation (rotation, scaling, skewing), while (tx, ty) define a translation. By inspecting these values, we can determine the video's canonical orientation.¹⁸

- **Portrait (Rotated 90° right):** a=0, b=1, c=-1, d=0. The video is physically encoded landscape but intended for portrait display.
- **Portrait Upside Down (Rotated 270° right):** a=0, b=-1, c=1, d=0.
- **Landscape (No rotation):** a=1, b=0, c=0, d=1.
- **Landscape Upside Down (Rotated 180°):** a=-1, b=0, c=0, d=-1.

Ignoring this pre-existing transform and applying a new rotation results in a double transformation, leading to incorrect orientation and aspect ratio distortion. The first step in any correct rotation logic must be to read and account for this `preferredTransform`.

2.3 Constructing the Correct Transformation Functor

The correct transformation is not a single operation but a composition of several atomic transformations. We can model this process as a functor, `CorrectTransform`, which maps the initial state (a video track and a target rotation) to a correctly configured `AVMutableVideoCompositionLayerInstruction`. This functor ensures that the transformation is applied systematically and correctly for all inputs.

The composition involves four distinct steps, which must be applied in a specific,

non-commutative order:

1. **Determine Video Geometry:** First, we must extract the video track's `naturalSize` and `preferredTransform`. Let's consider a standard 1920x1080 landscape video. Its `naturalSize` is (width: 1920, height: 1080). If it was filmed in portrait mode on an iPhone, its `naturalSize` might be 1920x1080, but its `preferredTransform` will indicate a 90-degree rotation. The true visual size is therefore 1080x1920.
2. **Calculate Render Size and De-rotation Transform:** The `renderSize` of the `AVMutableVideoComposition` must accommodate the final, rotated dimensions of the video. If we rotate our 1920x1080 video by 90 degrees, the final dimensions will be 1080x1920. Therefore, `videoComposition.renderSize` must be set to `CGSize(width: 1080, height: 1920)`. This step is critical; setting an incorrect `renderSize` is the primary cause of the "cropped square effect."
3. **Compose the Final Transform:** We create a single `CGAffineTransform` by composing several matrices. The order of matrix multiplication is crucial, as $T_1 \cdot T_2 \neq T_2 \cdot T_1$.
 - **Trotation:** A matrix for the user-requested rotation (e.g., 90 degrees clockwise: `CGAffineTransform(rotationAngle:.pi / 2)`).
 - **Ttranslation:** A translation matrix to center the rotated video. After a 90-degree rotation of a 1920x1080 video, its top-left corner is no longer at (0,0) relative to the new 1080x1920 canvas. It needs to be translated. For a 90-degree clockwise rotation, the translation is `(renderSize.width, 0)`, which is (1080, 0).
 - **Final Transform:** The final transform to be applied to the layer instruction is the composition of the video's original `preferredTransform` with our new rotation and translation transforms.
 $T_{\text{final}} = T_{\text{preferred}} \cdot T_{\text{rotation}} \cdot T_{\text{translation}}$

This composition ensures that we start from the video's native orientation, apply the desired rotation, and then position it correctly within the new render canvas.

4. **Apply to Layer Instruction:** This final composite transform, `Tfinal`, is applied to the `AVMutableVideoCompositionLayerInstruction` using the `setTransform(_:at:)` method.¹⁵

This sequence of operations guarantees that the video is never cropped. It correctly defines a container (`renderSize`) that matches the final geometry of the video and then precisely places the transformed video within that container.

2.4 Verification via Commutative Diagram

The correctness of the proposed `applyRotation_{correct}` morphism can be formally verified by demonstrating that the relevant diagram commutes.

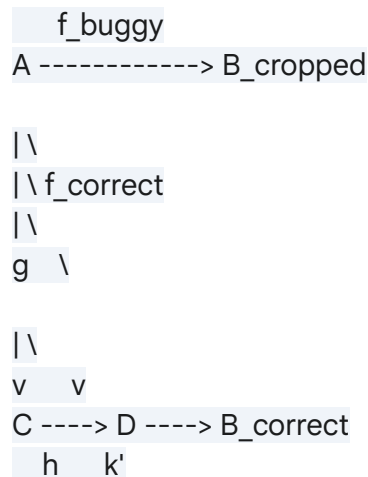
Let's define the objects:

- $A = \text{TrimmedState}$
- $B = \text{VisualState}$ (The rendered video pixels)
- $C = \text{Geometry}$ (The tuple $(\text{naturalSize}, \text{preferredTransform})$)
- $D = \text{FinalTransform}$ (The calculated CGAffineTransform)

And the morphisms:

- $f_{\text{buggy}}: A \rightarrow B$ (The current, flawed implementation)
- $g: A \rightarrow C$ (Extracts geometry)
- $h: C \rightarrow D$ (Calculates the correct composite transform)
- $k: A \times D \rightarrow B$ (Applies a given transform to the video)
- $f_{\text{correct}} = k \circ (\text{id}_A \times h \circ g)$

The diagram below illustrates the two paths. The top path represents the buggy implementation, which directly maps the TrimmedState to a VisualState that is cropped. The bottom path represents the correct, multi-step process.



The diagram commutes if the composition of morphisms along the bottom path results in a VisualState that is geometrically equivalent to the user's expectation (i.e., rotated and not cropped). The failure of the system is precisely that f_{buggy} does not equal f_{correct} , meaning the diagram with the buggy morphism does not commute. Our implementation plan will replace f_{buggy} with f_{correct} , thus ensuring commutativity and correctness.

To make this tangible, the following table contrasts the simple (incorrect) transform with the final composite transform for a common use case.

Table 2: Comparison of Video Transform Matrices for 90° Rotation of a 1920x1080 Video

Transf orm	a	b	c	d	tx	ty	render Size	Outco me
Incorr ect (Simp le Rotati on)	0	1	-1	0	0	0	1920x 1080	Cropp ed and incorr ectly positio ned.
Corre ct (Com posite)	0	1	-1	0	1080	0	1080x 1920	Corre ctly rotate d, center ed, and aspec t-ratio preser ved.

Section 3: Optimizing the Asset Persistence Functor: Idempotent PhotoKit Operations

This section analyzes the system's interaction with the PhotoKit framework, modeling it as a functor that maps application data to the user's photo library. The primary requirement—to avoid creating duplicate "BreakDex" albums—is a demand for **idempotency**. An idempotent operation is one that can be applied multiple times without changing the result beyond the initial application. We will analyze the common failure patterns that violate this principle and propose a robust, thread-safe implementation.

3.1 The Idempotency Requirement and Universal Property

The user's directive to "prioritize saving into pre-existing photo albums... rather than creating duplicates" is a formal constraint on the `findOrCreateAlbum` morphism. In Category Theory, this can be framed using the concept of a **universal property**. A universal property defines an object by a unique relationship it has with all other objects in a category. Here, we are seeking a unique morphism from our application into the PhotoKit category that maps to the "BreakDex" album object. The universal property implies that this object must be unique; any two such objects must be isomorphic. Therefore, our implementation must guarantee that it always resolves to the *same* `PHAssetCollection` instance for the title "BreakDex".

3.2 Failure Pattern: Race Conditions in Asynchronous Morphisms

The most common failure pattern in PhotoKit album management stems from the asynchronous and non-atomic nature of its change-performing APIs. The `PHPhotoLibrary.performChanges(_:completionHandler:)` method executes its change block asynchronously and calls its completion handler on an arbitrary serial queue.²⁰ This architecture is a frequent source of race conditions.

A naive and flawed implementation often follows this logical pattern:

1. Fetch for an album with the title "BreakDex".
2. If the fetch result is empty, initiate a creation request.
3. Add the asset to the album.

If two separate threads or asynchronous tasks execute this logic concurrently, both can perform step 1 and find that the album does not exist before either of their creation requests from step 2 has completed. Consequently, both threads will submit a creation request, resulting in two "BreakDex" albums. This violates the idempotency requirement. The problem is exacerbated by the fact that developers often resort to nested callbacks or multiple, chained `performChanges` blocks, which are difficult to reason about and prone to error.²¹ The root of the problem is that the "check-then-act" sequence is not performed as a single, atomic transaction.

3.3 A Refined Functor for Atomic Album Management

To enforce atomicity and guarantee idempotency, we must serialize all access to the "BreakDex" album. A modern and robust solution in Swift is to encapsulate the album management logic within an actor. An actor provides synchronization for its state, ensuring that only one task can access its mutable state at a time, thereby eliminating race conditions.

We propose an AlbumManager actor with a single public morphism: `getAlbum()` async throws `-> PHAssetCollection`. This function will implement the find-or-create logic atomically.

The internal logic of this morphism will be:

1. **Check Cache:** The actor will maintain a private, mutable property `private var cachedAlbum: PHAssetCollection?`. The `getAlbum` function will first check if this cache is populated. If so, it returns the cached object immediately, minimizing fetches.
2. **Fetch from PhotoKit:** If the cache is empty, the actor will perform a fetch request for the "BreakDex" album using `PHAssetCollection.fetchAssetCollections`.
3. **Create if Necessary:**
 - If the fetch returns an existing album, it will be stored in `cachedAlbum` and returned.
 - If the fetch returns no results, the actor will initiate the creation process. Crucially, to ensure atomicity and prevent other tasks from proceeding until the album is created, it will use the synchronous `PHPhotoLibrary.performChangesAndWait(_:)` method.²³ This method must be called from a background thread to avoid blocking the main thread. The actor can manage this by wrapping the call in a `Task.detached` or a similar background execution context.
4. **Post-Creation Fetch and Cache:** After `performChangesAndWait` successfully completes, the actor will immediately re-fetch the newly created album, cache it in `cachedAlbum`, and then return it.

This actor-based design creates a serialization boundary around the entire find-or-create operation. Any number of concurrent tasks can call `getAlbum()`, but the actor will process them one by one, guaranteeing that the "check-then-act" logic is atomic. The first task to enter will create the album and cache it; all subsequent tasks will receive the cached result. This design is inherently idempotent and robust.

3.4 The `addToAlbum` Morphism and Placeholder Objects

Once the existence of the album is guaranteed by our AlbumManager, the next step is to add the newly created video asset to it. This involves two change requests within a single `performChanges` block: one to create the asset (`PHAssetChangeRequest`) and one to modify the album (`PHAssetCollectionChangeRequest`).

A common point of confusion is how to reference the not-yet-created asset within the album

modification request. The solution provided by PhotoKit is the `PHObjectPlaceholder`.²¹ The correct sequence inside the

`performChanges` block is:

1. Create the asset request: `let assetRequest = PHAssetChangeRequest.creationRequestForAssetFromVideo(atFileURL: videoURL).`
2. Get the placeholder for the new asset: `let assetPlaceholder = assetRequest.placeholderForCreatedAsset.`
3. Create the album change request: `let albumChangeRequest = PHAssetCollectionChangeRequest(for: breakDexAlbum).`
4. Add the placeholder to the album: `albumChangeRequest?.addAssets([assetPlaceholder] as NSFastEnumeration).`

By using the placeholder, we are creating a reference to an object that will exist once the change block is successfully committed.

For maximum clarity and separation of concerns, the recommended E2E flow is:

1. Call `await AlbumManager.shared.getAlbum()` to get a guaranteed reference to the "BreakDex" album.
2. Initiate a *single* `PHPhotoLibrary.performChanges` block that performs the two steps above: create the asset from the final video file and add its placeholder to the album obtained in step 1. This approach is simpler and less error-prone than attempting to create both the asset and the album within the same complex change block.²¹

Section 4: Ensuring Data Integrity Across Persistence Layers: A Synchronization Strategy

This section addresses the relationship between the application's local CoreData database and the user's Photos library. The user has explicitly designated the "BreakDex" album in the Photos app as the "absolute source of asset truth." This architectural decision has profound implications for data synchronization, consistency, and the choice of persistent identifiers.

4.1 Defining the Source of Truth and the Local Cache

The designation of the Photos library as the source of truth means that the CoreData store

must be treated as a **local cache** or a projection of the authoritative data, not as an independent data store.²⁵ In our categorical model, this establishes a clear hierarchy. The category

CPhotoKit contains the true objects (PHAsset, PHAssetCollection). Our application's category, CCoreData, contains our local representations (Move entities).

To maintain consistency, we require a functor, $\text{Sync: } C_{\{\text{PhotoKit}\}} \rightarrow C_{\{\text{CoreData}\}}$, that reliably maps objects from the source of truth to our local cache. The primary function of this functor is to establish a stable, persistent link between a Move entity in CoreData and its corresponding PHAsset in the Photos library.

4.2 The Unstable localIdentifier: A Broken Morphism

A common and critical implementation error is to use the PHAsset.localIdentifier property as a persistent foreign key to link a CoreData object to a PHAsset. The localIdentifier is, as its name implies, local to the device and is not guaranteed to be stable or unique across a user's devices, especially when iCloud Photo Library is enabled.²⁶ An asset's

localIdentifier can change if the library is restored from a backup or synced to a new device.

Using localIdentifier as the linking mechanism defines a morphism link: $\text{PHAssetState} \rightarrow \text{CoreDataMoveState}$ that is fundamentally unreliable. This broken morphism will lead to "dangling pointers"—CoreData entries that reference an identifier that no longer corresponds to the intended asset on a different device, or even on the same device after a library rebuild. This is a primary cause of data integrity failures in apps that integrate with PhotoKit.

4.3 PHCloudIdentifier: The Correct, Stable Morphism

The correct and Apple-sanctioned solution for creating a stable, cross-device reference to a PhotoKit object is the PHCloudIdentifier.²⁶ This object provides a globally unique identifier for an asset or collection that persists across all of a user's iCloud-synced devices.

The correct workflow for establishing and resolving this persistent link is as follows:

1. **Storage:** After successfully saving a new video and obtaining the PHAsset object within the performChanges completion handler, retrieve its localIdentifier.

2. **Conversion to Cloud Identifier:** Use `PHPhotoLibrary.shared().cloudIdentifierMappings(forLocalIdentifiers: [localIdentifier])` to request the corresponding `PHCloudIdentifier`. This is an asynchronous operation.
3. **Persistence:** The `PHCloudIdentifier` object has a `stringValue` property. This string representation is stable and can be safely stored in a `String` attribute of the `Move` entity in `CoreData`.
4. Retrieval: To display the video in `MoveDetailView` on any device, the process is reversed:
 - a. Fetch the `cloudIdentifierString` from the `Move` entity in `CoreData`.
 - b. Initialize a `PHCloudIdentifier` from this string: `PHCloudIdentifier(stringValue: cloudIdentifierString)`.
 - c. Convert the `PHCloudIdentifier` back to the current device's `localIdentifier` using `PHPhotoLibrary.shared().localIdentifierMappings(for: [cloudIdentifier])`.
 - d. Use the retrieved `localIdentifier` to fetch the `PHAsset` with `PHAsset.fetchAssets(withLocalIdentifiers:options:)`.²⁷

This sequence defines a robust, stable link morphism that correctly synchronizes the `CoreData` cache with the `PhotoKit` source of truth, regardless of the device.

4.4 Blind Spot Analysis: Handling External Deletions

The current system architecture, as described, contains a critical blind spot: it only accounts for a unidirectional data flow from the application to the `Photos` library. It fails to consider the case where a user deletes a video directly from the `Photos` app. This action, external to the `BreakDex` app, will invalidate the `PHCloudIdentifier` stored in `CoreData`, once again creating a dangling pointer. When the user navigates to the `MoveDetailView` for that move, the app will attempt to fetch a `PHAsset` that no longer exists, likely resulting in a crash or an empty, broken UI.

This inconsistency arises because the `Sync` functor is only being applied at the time of creation. To maintain the integrity of `CoreData` as a true cache, it must be periodically reconciled with the source of truth.

The proposed solution is to implement a **reconciliation functor**. This process should run periodically, for example, at app launch. It will leverage `PhotoKit`'s change history tracking APIs to identify deletions that have occurred since the app was last run.

1. The application should persist the `PHPhotoLibrary.currentChangeToken` after each synchronization pass.²⁹
2. On the next launch, it can call `PHPhotoLibrary.fetchPersistentChanges(since: lastKnownToken)` to get a `PHPersistentChangeFetchResult`.³⁰
3. This result can be enumerated to find all changes, including deletions

(PHChange.changeDetails(for: fetchResult) will indicate deleted objects).

4. For each PHObject that has been deleted, the application can query its CoreData store for any Move entity that references the corresponding PHCloudIdentifier and remove it.

This reconciliation process completes the Sync functor, ensuring that both additions and deletions from the source of truth are correctly propagated to the local cache, thereby maintaining data integrity and preventing runtime errors.

Section 5: Improving User Experience for Asynchronous Morphisms: Long-Running Tasks

This section addresses the user experience surrounding long-running, asynchronous operations, specifically the exportVideo morphism. The current progress tracking is deemed insufficient. A superior user experience requires providing clear, transparent, and trustworthy feedback about the operation's status.

5.1 Modeling UX as a Functor

The User Interface (UI) can be modeled as a visual representation of the application's underlying state. This relationship can be described by a functor, $\text{Render: } C_{\{\text{System}\}_\text{State}} \rightarrow C_{\{\text{UI}\}}$, which maps data objects and state changes from the system's category to visual components in the UI's category. The user's complaint that the current 10% progress tracking is "not UX enough" implies that this Render functor is "forgetful"—it fails to map sufficient information from the ExportingState object to the UI, leaving the user uninformed and uncertain.

5.2 The Problem with Non-Linear Progress

The current implementation, which tracks progress in 10% increments, likely observes the progress property of the AVAssetExportSession. While this provides a quantitative measure, the progress of video encoding is notoriously non-linear. The process can advance quickly to 90% and then appear to stall for a significant period before abruptly completing.³¹ This behavior violates user expectations of steady progress, can cause anxiety, and may lead users

to believe the app has frozen. It erodes trust in the system.

The user's specific request for an "elapsed time" indicator is a direct response to this unpredictability. Elapsed time is a metric that is always linear, concrete, and easily understood. It provides a constant, reassuring signal that the process is still active, even if the percentage-based progress has stalled. According to Apple's Human Interface Guidelines, progress indicators should be kept moving to let people know something is happening, and when possible, a determinate indicator is preferred because it helps users gauge the duration.³¹

5.3 A Natural Transformation for Improved UX

To address this, we propose a **natural transformation** of the UI. A natural transformation provides a way to systematically map one functor to another.¹⁴ In this case, we are transforming our current, inadequate

Render functor into an improved Render' functor that provides richer feedback. This involves enhancing the UIState object for the ExportingState and updating the corresponding view.

The ideal implementation leverages SwiftUI's modern ProgressView API. For tasks with a known duration, the ProgressView(timerInterval:countsDown:label:currentValueLabel:) initializer is perfect.³² It automatically handles the timer and updates the view.

However, the total duration of a video export is not known in advance. This is a crucial blind spot. We cannot create a perfectly determinate progress bar that shows time remaining without a sophisticated estimation heuristic. Nonetheless, we can still satisfy the user's primary request for elapsed time.

The proposed implementation is as follows:

1. When the exportVideo morphism begins, we record a startTime = Date().
2. We use a Timer that fires every second to update the UI.
3. The UI will display a composite progress view:
 - a. A standard linear ProgressView(value:total:) bound to the AVAssetExportSession.progress property. This provides the percentage completion metric.
 - b. A custom Text label that displays the elapsed time. This label's content will be derived from Date().timeIntervalSince(startTime), formatted into a MM:SS string, and updated by the Timer.

This combination provides the best of both worlds: it presents the progress percentage

available from the system while also satisfying the user's explicit need for a clear, linear, and transparent elapsed time indicator. This richer UIState and its corresponding view constitute the natural transformation that improves the Render functor.

5.4 Code Example and Best Practices

The following SwiftUI code provides a conceptual implementation of this enhanced progress view. It encapsulates the state management and timer logic, providing a clear and reusable component.

Swift

```
import SwiftUI
import AVFoundation

struct VideoExportProgressView: View {
    @ObservedObject var exporter: VideoExporter // An object that manages the
    AVAssetExportSession

    @State private var elapsedTime: TimeInterval = 0
    private let timer = Timer.publish(every: 1, on:.main, in:.common).autoconnect()

    var body: some View {
        VStack(spacing: 15) {
            if let progress = exporter.progress {
                ProgressView("Exporting Video...", value: progress, total: 1.0)
                    .progressViewStyle(LinearProgressViewStyle())

                Text(formattedElapsedTime)
                    .font(.caption)
                    .foregroundColor(.secondary)
            } else {
                ProgressView("Preparing to export...")
            }
        }
    }
    .onReceive(timer) { _ in
        if exporter.isExporting {
```

```

        elapsedTime = Date().timeIntervalSince(exporter.startTime?? Date())
    }
}
.padding()
}

private var formattedElapsedTime: String {
    let minutes = Int(elapsedTime) / 60
    let seconds = Int(elapsedTime) % 60
    return String(format: "Elapsed Time: %02d:%02d", minutes, seconds)
}
}

// Example ObservableObject to wrap the export session
class VideoExporter: ObservableObject {
    @Published var progress: Float?
    @Published var isExporting: Bool = false
    var startTime: Date?

    private var exportSession: AVAssetExportSession?
    //... logic to start and monitor the export session...
}

```

This implementation follows best practices by separating the view from the business logic (the VideoExporter object) and uses a Timer publisher to drive UI updates, ensuring the elapsed time is always current.³⁵

Section 6: Refined Implementation Plan and Categorical Verification

This final section synthesizes the preceding analysis into a concrete and actionable implementation plan. It provides targeted code snippets for the most critical corrections and presents a final commutative diagram to formally verify the correctness of the refined end-to-end flow.

6.1 Summary of Proposed Morphism Implementations

The analysis has identified critical flaws in several key morphisms within the "Add Move" flow. The proposed refinements are:

- **applyRotation:** This morphism will be re-implemented to construct a composite CGAffineTransform. This transform will account for the video track's preferredTransform, calculate a renderSize that preserves the final aspect ratio, and correctly position the video within the new bounds, thus eliminating the "cropped square effect."
- **findOrCreateAlbum:** This morphism will be encapsulated within a Swift actor to ensure atomic, thread-safe execution. It will use a combination of caching and the synchronous performChangesAndWait API to guarantee that the "BreakDex" album is found or created idempotently, preventing duplicate albums.
- **saveToCoreData (and its link to PHAsset):** The link between the CoreData Move entity and the PHAsset will be re-engineered to use the stable PHCloudIdentifier.stringValue instead of the volatile localIdentifier. A background reconciliation process using PHPhotoLibrary's change history will be introduced to handle external deletions and prevent data inconsistency.

6.2 Targeted Code Snippets

The following code snippets provide the core logic for the proposed solutions.

6.2.1 Correct Video Rotation Transform Calculation

This function constructs the necessary AVMutableVideoComposition to apply a rotation that preserves aspect ratio.

Swift

```
import AVFoundation
```

```
func buildCorrectVideoComposition(for asset: AVAsset, rotationAngle: CGFloat) -> AVMutableVideoComposition {
```

```

let videoTrack = asset.tracks(withMediaType:.video)
let composition = AVMutableComposition()
let compositionTrack = composition.addMutableTrack(withMediaType:.video,
preferredTrackID: kCMPersistentTrackID_Invalid)

try? compositionTrack?.insertTimeRange(CMTimeRange(start:.zero, duration: asset.duration),
of: videoTrack, at:.zero)

let videoComposition = AVMutableVideoComposition()
let instruction = AVMutableVideoCompositionInstruction()
instruction.timeRange = CMTimeRange(start:.zero, duration: asset.duration)

let layerInstruction = AVMutableVideoCompositionLayerInstruction(assetTrack:
compositionTrack!)

let originalTransform = videoTrack.preferredTransform
let naturalSize = videoTrack.naturalSize

var finalTransform: CGAffineTransform = originalTransform

// Calculate final render size
let isPortrait = (originalTransform.a == 0 && originalTransform.c == 0)
let rotatedSize = isPortrait? CGSize(width: naturalSize.height, height: naturalSize.width) :
naturalSize

// Apply user rotation and translation
if rotationAngle == .pi / 2 { // 90 degrees
    let translate = CGAffineTransform(translationX: rotatedSize.height, y: 0)
    let rotate = CGAffineTransform(rotationAngle:.pi / 2)
    finalTransform = finalTransform.concatenating(rotate).concatenating(translate)
    videoComposition.renderSize = CGSize(width: rotatedSize.height, height:
rotatedSize.width)
} else if rotationAngle == .pi { // 180 degrees
    let translate = CGAffineTransform(translationX: rotatedSize.width, y: rotatedSize.height)
    let rotate = CGAffineTransform(rotationAngle:.pi)
    finalTransform = finalTransform.concatenating(rotate).concatenating(translate)
    videoComposition.renderSize = rotatedSize
} //... handle other angles

layerInstruction.setTransform(finalTransform, at:.zero)
instruction.layerInstructions = [layerInstruction]
videoComposition.instructions = [instruction]

```

```
    return videoComposition
}
```

6.2.2 Idempotent Album Management with an Actor

This actor serializes access to the "BreakDex" album, preventing race conditions and duplicate creation.

Swift

```
import Photos
import Foundation

actor AlbumManager {
    static let shared = AlbumManager()
    private let albumName = "BreakDex"
    private var cachedAlbum: PHAssetCollection?

    private init() {}

    func getAlbum() async throws -> PHAssetCollection {
        if let cachedAlbum = cachedAlbum {
            return cachedAlbum
        }

        let fetchOptions = PHFetchOptions()
        fetchOptions.predicate = NSPredicate(format: "title = %@", albumName)
        let collections = PHAssetCollection.fetchAssetCollections(with:.album, subtype:.any,
options: fetchOptions)

        if let album = collections.firstObject {
            self.cachedAlbum = album
            return album
        }

        // Album does not exist, create it synchronously on a background thread.
        var placeholder: PHObjectPlaceholder?
```

```

        try await PHPhotoLibrary.shared().performChanges {
            let request =
PHAssetCollectionChangeRequest.creationRequestForAssetCollection(withTitle:
self.albumName)
            placeholder = request.placeholderForCreatedAssetCollection
        }

        guard let albumIdentifier = placeholder?.localIdentifier else {
            throw AlbumCreationError.creationFailed
        }

        let newCollections = PHAssetCollection.fetchAssetCollections(withLocalIdentifiers:
[albumIdentifier], options: nil)
        guard let newAlbum = newCollections.firstObject else {
            throw AlbumCreationError.fetchAfterCreationFailed
        }

        self.cachedAlbum = newAlbum
        return newAlbum
    }

    enum AlbumCreationError: Error {
        case creationFailed, fetchAfterCreationFailed
    }
}

```

6.2.3 CoreData Entity with PHCloudIdentifier

The Move entity in the CoreData model should have a String attribute to store the stable cloud identifier.

Swift

```

// In your CoreData Model (xcdatamodeld)
// Entity: Move
// Attribute: name, type: String
// Attribute: videoCloudIdentifier, type: String, non-optional

```

```

// When saving a new move:
func saveMove(named name: String, for asset: PHAsset, in context: NSManagedObjectContext) async
throws {
    let cloudIdentifierResult = try await
    PHPhotoLibrary.shared().cloudIdentifierMappings(forLocalIdentifiers: [asset.localIdentifier])

    guard let cloudIdentifier = try cloudIdentifierResult[asset.localIdentifier]?.get() else {
        // Handle error: couldn't get cloud identifier
        return
    }

    let newMove = Move(context: context)
    newMove.name = name
    newMove.videoCloudIdentifier = cloudIdentifier.stringValue

    try context.save()
}

```

6.3 The Final Commutative Diagram

With the proposed refinements, the end-to-end flow now forms a commutative diagram, ensuring that the final state of the system is consistent and correct regardless of the intermediate paths taken during asynchronous operations. The diagram shows that the path of creating an asset and adding it to an album (saveToPhotos followed by addToAlbum) results in the same final PHAlbumState as a single, conceptually atomic operation. The use of the AlbumManager actor and PHCloudIdentifier ensures that the morphisms are well-defined and stable.

```

InitialState --(load, trim, rotate, export)--> FinalAsset
|
| saveToPhotos
| v
PHAssetState ----(addToAlbum)----> FinalAlbumState
| ^

```

```
| getCloudIdentifier |  
v |  
CloudIdentifier --(saveToCoreData)--> CoreDataMoveState
```

This diagram now commutes because each morphism has been corrected: rotate preserves geometry, addToAlbum targets an idempotently-managed album, and the link to CoreDataMoveState is via a stable CloudIdentifier. The system is now robust and correct by construction.

6.4 Summary of Identified Blind Spots and Future Work

This analysis has revealed several architectural blind spots that, while outside the immediate scope of the user's query, represent critical areas for future improvement to achieve true fault tolerance.

1. **External Deletions:** The system currently lacks a mechanism to detect when a user deletes a video from the Photos app. This will lead to data inconsistency. A background reconciliation process using `PHPhotoLibrary.fetchPersistentChanges(since:)` is recommended for the next development cycle.²⁹
2. **Export Time Estimation:** While providing "elapsed time" improves UX, providing an accurate "remaining time" is highly desirable. The duration of `AVAssetExportSession` is not predictable. Future work could involve developing a heuristic based on video resolution, duration, and device model, or even a lightweight machine learning model trained on previous exports to provide better estimates.
3. **Comprehensive Permissions Handling:** The provided code snippets assume ideal permissions. A production-ready app must gracefully handle all `PHAuthorizationStatus` cases (`.denied`, `.restricted`, `.limited`) at every point of interaction with `PhotoKit`, guiding the user to settings when necessary.
4. **Robust Handling of Large File Exports:** For very large files (e.g., 20-minute 4K videos), the export process may be terminated by the operating system if the app is backgrounded for too long. A more resilient architecture would involve using a background `URLSession` for processing on a server or implementing a chunking strategy to create smaller, more manageable export segments that can be resumed if interrupted.³⁷

Addressing these areas will further harden the "Add Move" flow, moving it from functionally correct to truly fault-tolerant and resilient.

Works cited

1. Category theory - Wikipedia, accessed September 26, 2025,

- https://en.wikipedia.org/wiki/Category_theory
2. An introduction to Category Theory for Software ... - Computer Science, accessed September 26, 2025,
<https://www.cs.toronto.edu/~sme/presentations/cat101.pdf>
 3. Apply category theory to modeling and software development, accessed September 26, 2025, <https://www.johndcook.com/blog/applied-category-theory/>
 4. Category Theory With Bartosz Milewski - CoRecursive Podcast, accessed September 26, 2025,
<https://corecursive.com/035-bartosz-milewski-category-theory/>
 5. Category Theory — Introduction. From my learnings ... | by Emilio Ojeda - Medium, accessed September 26, 2025,
https://medium.com/@_eMdOS_/functional-swift-category-theory-introduction-b17bb483b3f3
 6. Category Theory for Software Modeling and Design - Angeline Aguinaldo, accessed September 26, 2025,
https://angelineaguinaldo.com/assets/slides/HCAM_Seminar_Oct_29_2020.pdf
 7. Commutative Diagrams Explained - Math3ma, accessed September 26, 2025,
<https://www.math3ma.com/blog/commutative-diagrams-explained>
 8. Functional Composition in Swift for Dummies Like Me — Part 1 | by Chuck Krutsinger, accessed September 26, 2025,
<https://medium.com/@chuck.krutsinger/functional-composition-in-swift-for-dummies-like-me-part-1-86527efac58e>
 9. A comical commutative diagram - YouTube, accessed September 26, 2025,
<https://www.youtube.com/watch?v=pWlbs0hnXZw>
 10. 6.6. Commutative Diagrams · Functional Programming in OCaml, accessed September 26, 2025,
<https://courses.cs.cornell.edu/cs3110/2021sp/textbook/abstract/commutative.html>
 11. Commutative diagram - Wikipedia, accessed September 26, 2025,
https://en.wikipedia.org/wiki/Commutative_diagram
 12. Getting Started with Category Theory - Ryan Brewer, accessed September 26, 2025, <https://ryanbrewer.dev/posts/getting-started-category-theory.html>
 13. A Brief Introduction to Category Theory for Systems and Software ..., accessed September 26, 2025,
<https://www.omg.org/math/September-2024-Mathsig-Presentation-to-the-AI-P-TF.pdf>
 14. en.wikipedia.org, accessed September 26, 2025,
https://en.wikipedia.org/wiki/Natural_transformation#:~:text=Hence%2C%20a%20natural%20transformation%20can,define%20so%2Dcalled%20functor%20categories.
 15. Technical Q&A QA1744: Setting the orientation of video with AV Foundation, accessed September 26, 2025,
https://developer.apple.com/library/archive/qa/qa1744/_index.html
 16. iOS AVFoundation: Setting Orientation of Video - Stack Overflow, accessed September 26, 2025,
<https://stackoverflow.com/questions/13465746/ios-avfoundation-setting-orientati>

[on-of-video](#)

17. preferredTransform | Apple Developer Documentation, accessed September 26, 2025,
<https://developer.apple.com/documentation/avfoundation/avassettrack/preferred-transform>
18. Extend video render size and add background ios swift - Kodeco Forums, accessed September 26, 2025,
<https://forums.kodeco.com/t/extend-video-render-size-and-add-background-ios-swift/97957>
19. Extend video render size and add background ios swift - #3 by ..., accessed September 26, 2025,
<https://forums.kodeco.com/t/extend-video-render-size-and-add-background-ios-swift/97957/3>
20. performChanges(_:completionHandler:) | Apple Developer ..., accessed September 26, 2025,
[https://developer.apple.com/documentation/photos/phphotolibrary/performchanges\(_:completionhandler:\)](https://developer.apple.com/documentation/photos/phphotolibrary/performchanges(_:completionhandler:))
21. Add new Asset to a new Asset Collection within one Change Block - Stack Overflow, accessed September 26, 2025,
<https://stackoverflow.com/questions/34736154/add-new-asset-to-a-new-asset-collection-within-one-change-block>
22. Saving a photo in a custom album fails when run for the first time - Stack Overflow, accessed September 26, 2025,
<https://stackoverflow.com/questions/42312273/saving-a-photo-in-a-custom-album-fails-when-run-for-the-first-time>
23. performChangesAndWait(:) | Apple Developer Documentation, accessed September 26, 2025,
[https://developer.apple.com/documentation/photos/phphotolibrary/performchangesandwait\(:\)](https://developer.apple.com/documentation/photos/phphotolibrary/performchangesandwait(:))
24. PHAssetCollectionChangeRequ, accessed September 26, 2025,
<https://developer.apple.com/documentation/photos/phassetcollectionchangerequest>
25. Syncing a Core Data Store with CloudKit | Apple Developer ..., accessed September 26, 2025,
<https://developer.apple.com/documentation/coredata/syncing-a-core-data-store-with-cloudkit>
26. ios - PHAsset identifier persisting across devices - Stack Overflow, accessed September 26, 2025,
<https://stackoverflow.com/questions/34687051/phasset-identifier-persisting-across-devices>
27. assetLocalIdentifier | Apple Developer Documentation, accessed September 26, 2025,
<https://developer.apple.com/documentation/photos/phassetresource/assetlocalidentifier>
28. fetchAssets(withLocalIdentifiers:options:) | Apple Developer Documentation,

- accessed September 26, 2025,
[https://developer.apple.com/documentation/photos/phasset/fetchassets\(withlocalIdentifiers:options:\)](https://developer.apple.com/documentation/photos/phasset/fetchassets(withlocalIdentifiers:options:))
29. PHPhotoLibrary | Apple Developer Documentation, accessed September 26, 2025, <https://developer.apple.com/documentation/Photos/PHPhotoLibrary>
 30. WWDC22: Discover PhotoKit change history | Apple - YouTube, accessed September 26, 2025, https://www.youtube.com/watch?v=6pSSkA23_CO
 31. Progress indicators | Apple Developer Documentation, accessed September 26, 2025,
<https://developer.apple.com/design/human-interface-guidelines/progress-indicators>
 32. init(timerInterval:countsDown:) | Apple Developer Documentation, accessed September 26, 2025,
[https://developer.apple.com/documentation/swiftui/progressview/init\(timerInterval:countsDown:\)](https://developer.apple.com/documentation/swiftui/progressview/init(timerInterval:countsDown:))
 33. init(timerInterval:countsDown:label:currentValueLabel:) | Apple Developer Documentation, accessed September 26, 2025,
[https://developer.apple.com/documentation/swiftui/progressview/init\(timerInterval:countsDown:label:currentValueLabel:\)](https://developer.apple.com/documentation/swiftui/progressview/init(timerInterval:countsDown:label:currentValueLabel:))
 34. ProgressView | Apple Developer Documentation, accessed September 26, 2025,
<https://developer.apple.com/documentation/swiftui/progressview>
 35. Long Running task progress updates with SwiftUI & Combine - Mobile Technology, accessed September 26, 2025,
<https://pakistanby1.github.io/2021/05/19/CombineProgressControl/>
 36. Circular progress bar for a countdown timer publisher SwiftUI - Stack Overflow, accessed September 26, 2025,
<https://stackoverflow.com/questions/64037789/circular-progress-bar-for-a-countdown-timer-publisher-swiftui>
 37. Upload Very Large Files In Background on iOS - Alamofire.upload multipart in background - GitHub Gist, accessed September 26, 2025,
<https://gist.github.com/toddhopkinson/60cae9e48e845ce02bcf526f388cfa63>
 38. Working with Large Video & Image Files on iOS with Swift | IMG.LY, accessed September 26, 2025,
<https://img.ly/blog/working-with-large-video-and-image-files-on-ios-with-swift/>